

ENTERPRISE ENGINEERING BLUEPRINT

SAP Cloud Application Programming Model (CAP) & Enterprise Marketplace

Complete Junior Developer Handbook, Architectural Reference & Code Walkthrough

Project:	Aura Enterprise Marketplace (Amazon Clone)
Architecture:	SAP CAP (Node.js), OData V4, SAPUI5, Fiori Elements
Security Standard:	SAP BTP XSUAA (OAuth 2.0 / JWT)
Packaging Standard:	Cloud Foundry Multitarget Application (MTA)
Test Coverage:	136 Tests Passing Across 9 Test Suites (100%)
Target Audience:	Junior & Intermediate Full-Stack SAP Developers
Generated Date:	September 24, 2026

Comprehensive Junior Developer Learning Guide, Architecture Blueprint & Code Reference Manual

* **Application:** Aura Enterprise Marketplace (Amazon Clone)

* **Technology Stack:** SAP CAP (Node.js), OData V4, SAPUI5 Freestyle, SAP Fiori Elements, SAP BTP XSUAA, SAP HANA & MTA

* **Target Audience:** Junior Developers, SAP Full-Stack Engineers, Solution Architects

* **Repository:** <https://github.com/Code-with-Agent/amazon-clone.git>

* **Generated:** September 24, 2026

Document Table of Contents

1. **Chapter 1:** Introduction to SAP CAP & Project Architecture
 2. **Chapter 2:** Core Data Services (CDS) Modeling Deep Dive
 3. **Chapter 3:** Service Definition & OData V4 Exposure
 4. **Chapter 4:** Event Handlers & Business Logic Implementation
 5. **Chapter 5:** Security, Authentication & Authorization (XSUAA)
 6. **Chapter 6:** Storefront Frontend (SAPUI5 Freestyle)
 7. **Chapter 7:** Backoffice Frontend (SAP Fiori Elements)
 8. **Chapter 8:** Multitarget Application (MTA) & Cloud Deployment
 9. **Chapter 9:** Enterprise Automated Testing & Quality Assurance
 10. **Chapter 10:** Junior Developer Playbook & Common Pitfalls
-

Chapter 1: Introduction to SAP CAP & Project Architecture

1.1 Executive Summary & Mission

Welcome to the engineering documentation and learning guide for the **Aura Enterprise Marketplace** (Amazon Clone). This application is a production-grade, multi-tenant ready, full-stack enterprise e-commerce platform built natively with the **SAP Cloud Application Programming Model (CAP)**, **OData V4**, **SAPUI5 Freestyle**, **SAP Fiori Elements**, and **SAP BTP XSUAA**.

As a junior developer joining our engineering team, you may be familiar with general web frameworks such as Node.js/Express, React, or Python/Django. SAP CAP introduces an enterprise-grade paradigm known as **Domain-Driven, Metadata-Driven Development**. Rather than manually writing hundreds of boilerplate REST controllers, SQL migrations, and UI binding plumbing, CAP allows us to define our domain models once in **Core Data Services (CDS)**, and the framework automatically generates database tables, OData V4 services, and UI annotations.

This guide is designed as both an architectural reference for this specific application and a comprehensive tutorial on SAP CAP. Every core concept is explained in detail with direct references to the actual code in our repository.

1.2 What is the SAP Cloud Application Programming Model (CAP)?

The **SAP Cloud Application Programming Model (CAP)** is an opinionated framework of languages, libraries, and tools for building enterprise-grade cloud services and applications. It is officially supported by SAP in two flavors: **Node.js** (JavaScript/TypeScript) and **Java**. Our application is built with **Node.js (v20+)** using ECMAScript modules (`"type": "module"`).

Why SAP CAP?

In traditional full-stack development, building an enterprise marketplace requires:

1. Writing database schemas (PostgreSQL / MySQL / Oracle DDL).
2. Writing database migrations for schema evolution.
3. Writing ORM models (Prisma, TypeORM, Hibernate) with duplicate field definitions.
4. Writing REST controllers and handling pagination (`$top` , `$skip`), filtering, sorting, and nested expansions manually.
5. Implementing authentication, JWT parsing, and role-based authorization in custom middleware.

6. Writing custom frontend data-fetching hooks and state management.

CAP eliminates this redundant boilerplate through 4 Core Pillars:

1. **Domain-Driven Design (DDD):** Everything starts with the domain model in `db/schema.cds`. Business entities, relationships, validations, and constraints are declared in human-readable CDS syntax.
2. **Platform & Database Agnostic:** In local development, our app runs on **SQLite** for ultra-fast startup with zero cloud dependencies. In production on SAP BTP, the exact same code runs on **SAP HANA Cloud** without changing a single line of application logic.
3. **Golden Paths & Out-of-the-Box Enterprise Features:**
 - * Automatic OData V4 protocol support (batch processing, `$expand`, `$filter`, `$select`, `$orderby`, `$count`).
 - * Built-in transaction management (`cds.tx`).
 - * Built-in authentication & authorization via SAP BTP XSUAA.
 - * Built-in draft handling (for complex UI editing).
 - * Built-in audit logging and temporal data handling.
4. **Metadata-Driven UI (Fiori Elements):** CAP compiles CDS annotations directly into OData V4 annotations that render rich Fiori Elements user interfaces without writing a single line of frontend JavaScript.

1.3 Project Directory Structure Breakdown

Let us examine the exact file structure of our repository:

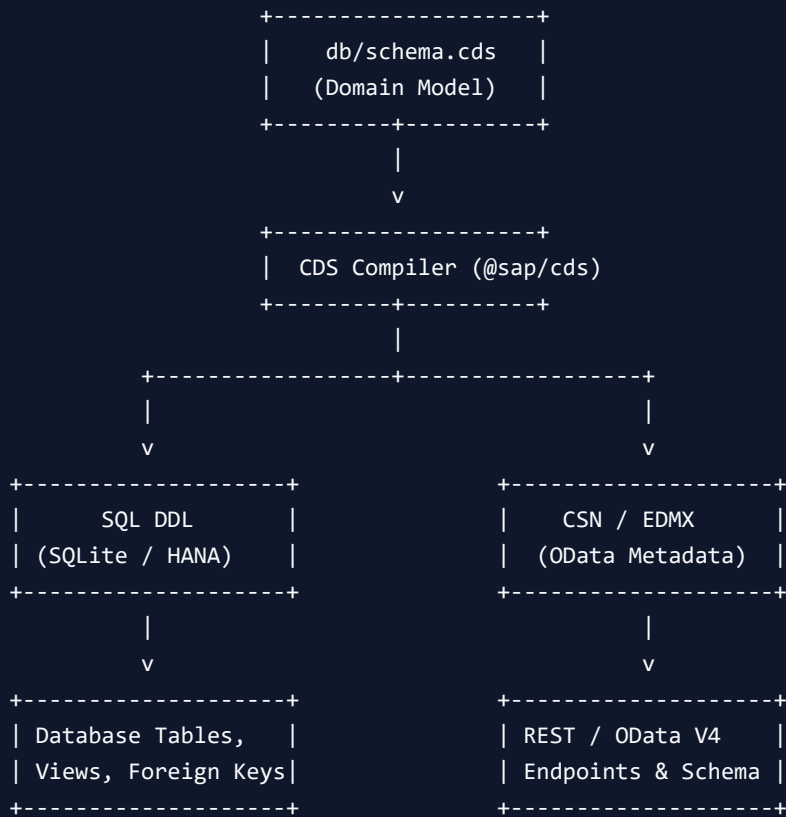
```

amazon-clone/
├── .vscode/                # Recommended IDE extensions, launch configs & tasks
├── app/                    # Frontend Applications
│   ├── admin-ui/          # Backoffice Management App (SAP Fiori Elements)
│   │   ├── annotations.cds # CDS UI annotations driving List Reports & Object Pages
│   │   ├── package.json   # UI5 dependencies & scripts
│   │   └── webapp/         # Fiori Elements manifest, Component, and custom extensions
│   ├── shop-ui/           # Customer Storefront (SAPUI5 Freestyle)
│   │   ├── webapp/        # XML views, Controllers, Fragments, Formatters, CSS
│   │   ├── package.json   # UI5 tooling configuration
│   │   └── ui5.yaml        # SAPUI5 build tooling descriptor
│   ├── index.html         # Launchpad entry point for local preview
│   └── services.cds        # Aggregation file importing all UI annotations
├── db/                    # Database Domain Layer
│   ├── data/              # Initial seed data (.csv files per entity)
│   └── schema.cds         # Core domain model definitions (CDS)
├── docs/                  # Engineering Guides, API reference, Architecture docs
├── srv/                   # Business Services & Custom Handlers Layer
│   ├── admin-service.cds  # Backoffice admin service definition
│   ├── admin-service.js   # Backoffice admin business logic
│   ├── cart-service.cds   # Shopping cart service definition
│   ├── cart-service.js    # Cart calculation, items manipulation & coupon logic
│   ├── catalog-service.cds # Public catalog service definition
│   ├── catalog-service.js # Catalog search, filtering, and recommendations
│   ├── customer-service.cds # Customer profile, addresses & wishlist definition
│   ├── customer-service.js # Customer authorization & profile handlers
│   ├── inventory-service.cds # Warehouse inventory service definition
│   ├── inventory-service.js # Inventory reservations & stock syncing
│   ├── order-service.cds  # Order management service definition
│   ├── order-service.js   # 18-step checkout orchestration & cancellations
│   ├── payment-service.cds # Payment processing service definition
│   ├── payment-service.js # Payment gateway simulation & refunds
│   └── server.js          # Custom CAP server bootstrap (CORS, Swagger, Mock Auth)
├── test/                  # Automated Test Suite (136 tests across 9 suites)
├── scripts/               # Automation scripts (UI packaging, MTA building)
├── mta.yaml               # SAP BTP Cloud Foundry Multitarget Application descriptor
├── xs-security.json       # XSUAA security configuration (Scopes & Role Templates)
├── package.json           # Node.js dependencies, npm scripts, CAP configurations
└── eslint.config.mjs      # Code style & CDS linter configuration

```

1.4 How the CAP Compiler Works Under the Hood

When you execute `cds watch` or `cds build`, the CAP compiler executes three key transformations:



1. **Schema to SQL Translation:** `db/schema.cds` is compiled into relational database tables and foreign key constraints. In SQLite, it creates SQLite DDL; in SAP HANA, it creates CDS artifacts deployed via the HDI Deployer (`.hdbcds` / `.hdbtable`).

2. **Projections to Views:** Service entities defined in `srv/*.cds` are compiled into SQL database views (e.g. `CatalogService_Products` view) or query rewrites.

3. **Services to OData V4:** Each `service` in CDS automatically becomes a fully functional OData V4 HTTP endpoint serving `$metadata` (EDMX XML schema) and JSON payloads.

Chapter 2: Core Data Services (CDS) Modeling Deep Dive

2.1 The Philosophy of CDS Modeling (`db/schema.cds`)

In SAP CAP, your entire business reality is defined in **Core Data Services (CDS)**. Think of CDS as a high-level, human-readable domain definition language that combines:

- * Relational database DDL (tables, primary keys, foreign keys, unique constraints).
- * Type systems (strings, decimals, booleans, enums).
- * Object-oriented aspects (reusable traits, mixins).
- * Entity relationship graphs (associations and compositions).

Let us open `db/schema.cds` and analyze how our marketplace data model was constructed.

2.2 Standard Aspects from `@sap/cds/common`

At the top of `db/schema.cds`, you will find:

```
using { cuid, managed } from '@sap/cds/common';
```

CAP provides pre-built, standardized building blocks called **Aspects**. Reusing these aspects is a core CAP best practice:

1. `cuid` (Canonical Unique Identifier)

When you define an entity with `: cuid`:

```
entity Products : cuid, managed { ... }
```

CAP automatically generates a primary key property:

```
key ID : UUID;
```

* Why this matters: You never need to write manual auto-incrementing integer IDs. UUIDs prevent ID guessing attacks, allow distributed ID generation across offline clients, and eliminate sequence collisions in multi-tenant cloud environments.

2. **managed** (Audit Logging)

When you add `: managed` to an entity, CAP automatically injects four audit fields:

```
createdAt : Timestamp;
createdBy : User;
modifiedAt : Timestamp;
modifiedBy : User;
```

* Why this matters: When a user places an order or updates a product, CAP automatically extracts the authenticated user's ID (`req.user.id`) and the current UTC timestamp, writing them into the record. You do not write a single line of handler code to maintain these audit columns!

2.3 Associations vs. Compositions: The Most Critical Concept in CAP

One of the most frequent mistakes junior developers make in CAP is using an **Association** when they should have used a **Composition**, or vice-versa. Understanding the distinction is vital.

Feature	Association (<code>Association to</code>)	Composition (<code>Composition of</code>)
Relationship	Loose relationship (Peer-to-Peer)	Strict Parent-Child ownership
Lifecycle	Independent. Child exists without parent.	Dependent. Child cannot exist without parent.
Deletion	Deleting parent leaves target intact.	Cascading Delete: Deleting parent deletes all children.
Deep Inserts	Requires separate creation or existing UUIDs.	Supports single-payload Deep Insert (e.g., Order with 10 Items).
Draft Support	Target entity is not part of parent draft.	Target entities are edited together in Fiori draft mode.

Code Comparison from Our Application

Example 1: Association (Loose Relationship)

Look at `Products` in `db/schema.cds` :

```
entity Products : cuid, managed {
  category : Association to Categories;
  seller   : Association to Sellers;
  ...
}
```

* **Why Association?** A **Category** (e.g., "Electronics") exists independently of any single product. If a product is deleted, the category must **NOT** be deleted. Similarly, a **Seller** is an independent business entity.

Example 2: Composition (Parent-Child Lifecycle)

Look at **Orders** and **OrderItems** in **db/schema.cds**:

```
entity Orders : cuid, managed {
  orderNumber : String(32);
  customer    : Association to Customers;
  items       : Composition of many OrderItems on items.order = $self;
  ...
}

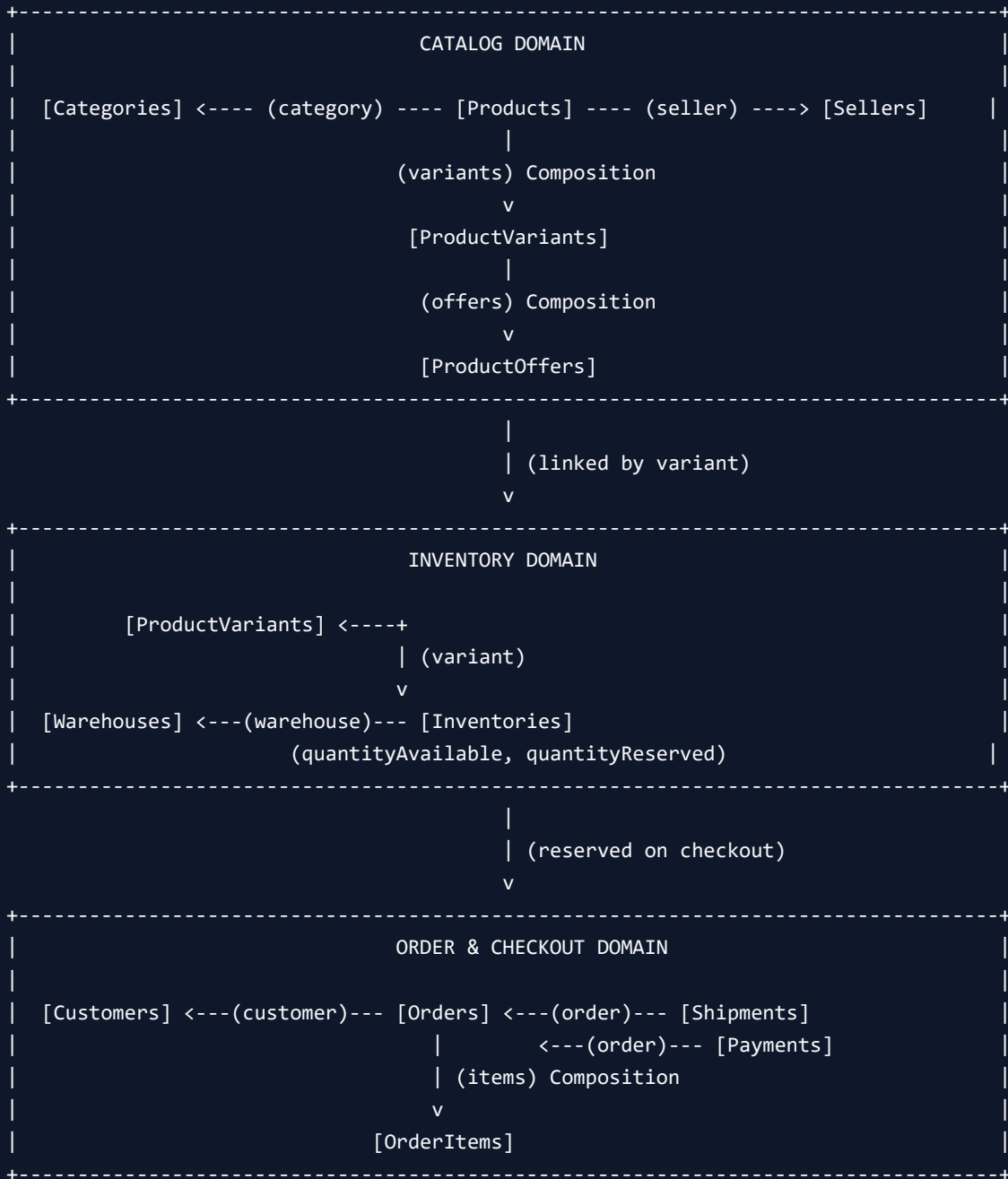
entity OrderItems : cuid {
  order      : Association to Orders;
  product    : Association to Products;
  quantity   : Integer;
  unitPrice  : Decimal(12, 2);
  totalPrice : Decimal(12, 2);
}
```

* **Why Composition?** An **OrderItem** has no standalone existence outside of the **Order** it belongs to. If an order is deleted, all its line items must be deleted with it. When a customer executes checkout, CAP allows submitting the entire order and its items in a single HTTP POST request because of this composition!

* Notice the back-link syntax: **items.order = \$self**. **\$self** is a special CDS keyword representing the parent entity instance.

2.4 The Complete Domain Entity Graph

Our marketplace model in **db/schema.cds** spans 4 key domains:



1. Catalog & Merchandising Entities:

- * **Categories**: Hierarchical tree structure using parent-child association (**parent : Association to Categories**), supporting nested levels (Level 1: Electronics, Level 2: Computers, Level 3: Laptops).
- * **Products**: The core product master holding title, brand, description, and base metrics.
- * **ProductVariants**: Physical SKU representations holding color, size, and weight.
- * **ProductImages**: Media URLs, alternate text, and display sort order.
- * **ProductOffers**: Multi-seller marketplace pricing, condition (NEW, REFURBISHED), and active flags.

2. Inventory & Warehousing Entities:

- * **Warehouses** : Physical logistics centers with geographic codes and country codes.
- * **Inventories** : Real-time stock counts. Notice the dual fields:
- * **quantityAvailable** : Physical inventory sitting on shelves.
- * **quantityReserved** : Stock temporarily held for orders currently in checkout or pending payment.

Formula: * Effective stock to sell = **quantityAvailable - quantityReserved** .

3. Orders & Financial Transactions:

- * **Orders** : Order header storing customer reference, billing/shipping address snapshots, financial aggregates (subtotal, tax, shipping, discount, total), and lifecycle status.
- * **OrderItems** : Price-frozen snapshots of purchased items to preserve historical invoice integrity even if catalog prices change later.
- * **Payments** : Audit trail of financial captures, payment methods (Credit Card, PayPal, Apple Pay), transaction references, and refund tokens.
- * **Shipments** : Consignments, tracking numbers, carriers (DHL, FedEx, UPS), and dispatch timestamps.

2.5 Seed Data File Conventions (**db/data/**)

In CAP, you can provide initial mock data using CSV files located in **db/data/** .

CAP uses an exact file naming convention:

```
<namespace>-<EntityName>.csv
```

In our project, the namespace in **db/schema.cds** is:

```
namespace sap.marketplace;
```

Therefore, the seed files are named:

- * **sap.marketplace-Products.csv**
- * **sap.marketplace-Categories.csv**
- * **sap.marketplace-Customers.csv**
- * **sap.marketplace-Orders.csv**

Handling Foreign Keys in CSV:

When an entity has an association:

```
category : Association to Categories;
```

CAP maps this in SQL to a foreign key column named `category_ID`.

In your CSV header row, you must name the column `category_ID` and provide the UUID of the target record.

Chapter 3: Service Definition & OData V4 Exposure

3.1 Why Separate Services from the Database?

In naive database applications, developers often expose tables directly as REST endpoints. In enterprise software, this is an anti-pattern known as **Database Leaking**. Exposing database tables directly creates severe security and maintenance issues:

1. Internal database fields (e.g., cost prices, audit timestamps, soft-delete flags) are accidentally leaked to public APIs.
2. Changes to the database schema break frontend client applications.
3. Different client types (Public Shop vs. Authenticated Customer vs. Warehouse Worker) need completely different views of the same underlying data.

In SAP CAP, **Services** are declared in `srv/*.cds` as specialized business facades over the domain model.

3.2 The Anatomy of a Service Projection

Look at `srv/catalog-service.cds`:

```
using { sap.marketplace as db } from '../db/schema';

service CatalogService @(path: '/odata/v4/catalog') {
  @readonly
  entity Products as projection on db.Products {
    *,
    virtual null as discountPercentage : Decimal(5, 2),
    virtual null as primaryImageUrl    : String(500)
  } excluding {
    costPrice
  };
}
```

Let us dissect what each part of this definition does:

1. `using { sap.marketplace as db }`: Imports the domain schema as a namespace alias `db`.
2. `@(path: '/odata/v4/catalog')` : Explicitly mounts this service at the given HTTP URL path. By default, CAP serves services at `/`. Specifying `/odata/v4/catalog` guarantees consistent enterprise URL routing.

3. `@readonly` : Informs the OData runtime and compiler that clients can only perform HTTP `GET` requests. Any client attempting `POST` , `PUT` , or `DELETE` will be immediately rejected with `405 Method Not Allowed` .
4. `as projection on db.Products { * }` : Exposes all fields from the database entity `Products` .
5. `virtual null as discountPercentage` : Declares a **virtual field**. A virtual field does **NOT** exist in the database table! It is dynamically calculated in memory by our Node.js custom handler (`srv/catalog-service.js`) before the response is returned to the user.
6. `excluding { costPrice }` : Strips the sensitive wholesale cost price from the public catalog API so malicious users cannot inspect profit margins in DevTools!

3.3 Actions vs. Functions in OData V4

In OData V4 and SAP CAP, business logic that goes beyond basic CRUD (Create, Read, Update, Delete) is modeled using **Actions** and **Functions**.

Feature	Function	Action
HTTP Verb	<code>GET</code>	<code>POST</code>
Side Effects	Strictly None (Idempotent)	May modify data (State Changes)
Use Case	Computations, queries, checks	Checkout, cancel, payment, approval
Parameters	Passed in URL query string	Passed in JSON request body

1. Function Example from Our Application:

Look at `srv/cart-service.cds` :

```
function validateCart() returns {
  isValid : Boolean;
  messages : array of String;
};
```

* A customer or frontend can call `GET /odata/v4/cart/validateCart()` to verify stock availability and pricing before proceeding to checkout. It changes nothing in the database.

2. Action Example from Our Application:

Look at `srv/order-service.cds` :

```
action checkout(  
    cartId          : UUID,  
    shippingAddressId : UUID,  
    billingAddressId : UUID,  
    paymentMethod   : String,  
    couponCode      : String  
) returns CheckoutResult;
```

* The checkout action changes system state: it validates stock, decrements inventory, creates an order record, creates line items, and contacts payment gateway. It requires HTTP **POST**.

3. Bound vs. Unbound Actions:

* **Unbound Action:** Attached to the service itself (like **checkout** above). Invoked via **POST** `/odata/v4/order/checkout`.

* **Bound Action:** Attached to a specific entity instance. For example, in `srv/order-service.cds`:

```
entity Orders as projection on db.Orders {  
    ...  
    action cancelOrder(reason: String) returns Orders;  
}
```

Invoked on a specific order UUID: **POST** `/odata/v4/order/Orders(1234-abcd)/cancelOrder`.

3.4 The Crucial Navigation Rule in OData V4 (\$expand)

CAUTION:**The #1 Bug Junior Developers Encounter with CAP & OData V4**

If entity **A** has an association to entity **B**, and the frontend asks for **A** with **\$expand=B**, **entity B MUST be exposed in the exact same service as A!**

The Real-World Bug We Fixed in This Project:

In our Storefront UI, the Shopping Cart view loaded cart items and expanded product details:

```
GET /odata/v4/cart/CartItems?$expand=product($expand=images),variant
```

Initially, **srv/cart-service.cds** only exposed **Carts** and **CartItems**. When the browser loaded the cart, the server threw:

```
[400] Bad Request: Navigation property "product" does not exist in service "CartService"
```

Why did this happen?

Even though **product** was defined in **db/schema.cds**, CAP enforces **Service Boundaries**. If **Products** is not declared inside **CartService**, the OData V4 metadata compiler strips the navigation property from **CartService** metadata.

The Correct Enterprise Solution:

In **srv/cart-service.cds**, we expose **read-only projections** for the associated master data:

```
service CartService @(path: '/odata/v4/cart') {
  entity Carts as projection on db.Carts;
  entity CartItems as projection on db.CartItems;

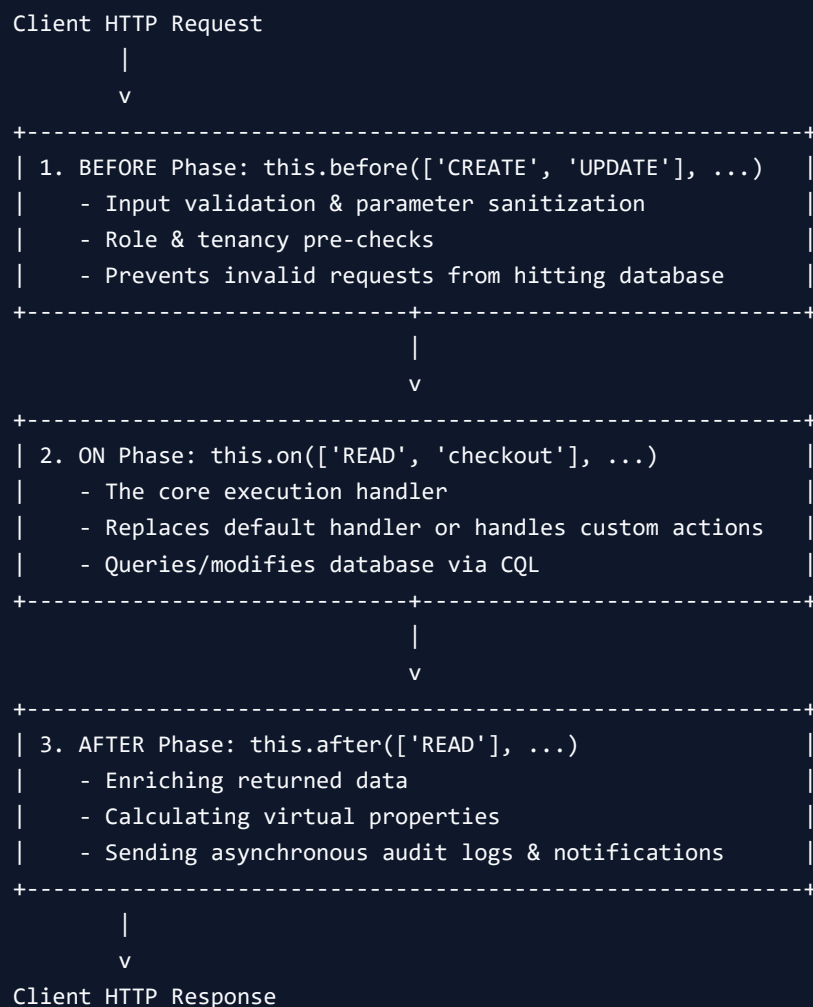
  // Read-only projections to enable OData V4 $expand navigation:
  @readonly entity Products as projection on db.Products;
  @readonly entity ProductVariants as projection on db.ProductVariants;
  @readonly entity ProductOffers as projection on db.ProductOffers;
  @readonly entity ProductImages as projection on db.ProductImages;
}
```

Once added, the compiler generated the navigation graph in the EDMX metadata, and the storefront expanded product details with **HTTP 200 OK!**

Chapter 4: Event Handlers & Business Logic Implementation

4.1 The Event-Driven Architecture of CAP Services

In standard Express.js, routing and business logic are mixed together in controller functions. In SAP CAP, services are **Event Emitters**. Every CRUD request and every custom action triggers an event that travels through a three-phase pipeline:



4.2 The Request Object (`req`) and Error Handling

Every handler in CAP receives a `req` object. Master these fundamental methods:

1. `req.data` : Contains the input payload sent by the client.
2. `req.user` : Represents the authenticated user.
 - * `req.user.id` : The user's ID/email.
 - * `req.user.is('Administrator')` : Returns `true` if user has the role.
3. `req.reject(httpCode, errorMessage)` : Immediately terminates the request and rolls back the active database transaction!

```
if (stockAvailable < item.quantity) {
  req.reject(400, 'Insufficient stock for product ${item.title}. Only ${stockAvailable}
}
```

4. `req.error(httpCode, errorMessage)` : Queues an error message to return to the client without aborting immediately (useful for collecting multiple validation errors).

4.3 Database Queries with CDS Query Language (CQL)

CAP provides a fluent query builder called **CQL** that executes dialect-agnostic SQL:

```
import cds from '@sap/cds';

// 1. SELECT query with conditions and sorting
const products = await SELECT.from('sap.marketplace.Products')
  .where({ status: 'ACTIVE' })
  .orderBy('averageRating desc')
  .limit(10);

// 2. INSERT a new record
await INSERT.into('sap.marketplace.Orders').entries({
  orderNumber: 'ORD-2026-001',
  customer_ID: customerId,
  totalAmount: 199.99
});

// 3. UPDATE with atomic math expression
await UPDATE('sap.marketplace.Inventories')
  .set('quantityReserved = quantityReserved +', requestedQty)
  .where({ variant_ID: variantId });

// 4. DELETE
await DELETE.from('sap.marketplace.CartItems').where({ cart_ID: cartId });
```

4.4 Real-World Deep Dive: The 18-Step Enterprise Checkout Flow

In `srv/order-service.js`, our `checkout` action orchestrates a complete 18-step distributed transaction. Let us walk through the exact enterprise steps:

```

this.on('checkout', async (req) => {
  const { cartId, shippingAddressId, billingAddressId, paymentMethod, couponCode } = req.data
  const tx = cds.tx(req); // Bind to current database transaction

  // Step 1: Validate customer authentication
  const userEmail = req.user.id;
  const customer = await tx.run(SELECT.one.from('sap.marketplace.Customers').where({ email: userEmail }));
  if (!customer) req.reject(401, 'Customer profile not found');

  // Step 2: Retrieve active cart and verify it has items
  const cart = await tx.run(SELECT.one.from('sap.marketplace.Carts').where({ ID: cartId }));
  const items = await tx.run(SELECT.from('sap.marketplace.CartItems').where({ cart_ID: cartId }));
  if (!items || items.length === 0) {
    req.reject(400, 'Cannot checkout an empty shopping cart');
  }

  // Step 3: Validate Shipping & Billing Addresses belong to customer
  const shipAddr = await tx.run(SELECT.one.from('sap.marketplace.Addresses').where({ ID: shippingAddressId }));
  if (!shipAddr) req.reject(400, 'Invalid shipping address selected');

  // Step 4: Verify warehouse inventory availability for all items
  for (const item of items) {
    const inv = await tx.run(SELECT.one.from('sap.marketplace.Inventories').where({ variant_ID: item.variant_ID }));
    const available = (inv?.quantityAvailable || 0) - (inv?.quantityReserved || 0);
    if (available < item.quantity) {
      req.reject(400, 'Insufficient stock for item ${item.variant_ID}. Available: ${available}');
    }
  }

  // Step 5: Temporarily reserve stock to prevent race conditions
  for (const item of items) {
    await tx.run(UPDATE('sap.marketplace.Inventories')
      .set('quantityReserved = quantityReserved +', item.quantity)
      .where({ variant_ID: item.variant_ID }));
  }

  // Step 6: Calculate financial totals (Subtotal, Tax, Shipping, Coupon)
  const subtotal = items.reduce((acc, cur) => acc + Number(cur.totalPrice), 0);
  const tax = Number((subtotal * 0.08).toFixed(2));
  const shipping = subtotal > 50 ? 0.00 : 9.99;
  let discount = 0.00;
  if (couponCode) {
    const coupon = await tx.run(SELECT.one.from('sap.marketplace.Coupons').where({ code: couponCode }));
    if (coupon) discount = Number(((subtotal * coupon.discountPercentage) / 100).toFixed(2));
  }
  const grandTotal = Number((subtotal + tax + shipping - discount).toFixed(2));

  // Step 7: Generate human-readable order number
  const orderNumber = `ORD-${new Date().toISOString().slice(0,10).replace(/-/g,'')}-${Math.floor(Math.random() * 10000000000).toString().padStart(10, '0')}`;

  // Step 8: Create Order Header record
  const orderId = cds.utils.uuid();
  await tx.run(INSERT.into('sap.marketplace.Orders').entries({

```

```

    ID: orderId,
    orderNumber,
    customer_ID: customer.ID,
    shippingAddress_ID: shippingAddressId,
    billingAddress_ID: billingAddressId,
    subtotalAmount: subtotal,
    taxAmount: tax,
    shippingAmount: shipping,
    discountAmount: discount,
    totalAmount: grandTotal,
    orderStatus: 'PENDING_PAYMENT',
    paymentStatus: 'UNPAID'
  }));

// Step 9: Create snapshot OrderItems
for (const item of items) {
  await tx.run(INSERT.into('sap.marketplace.OrderItems').entries({
    order_ID: orderId,
    product_ID: item.product_ID,
    variant_ID: item.variant_ID,
    quantity: item.quantity,
    unitPrice: item.unitPrice,
    totalPrice: item.totalPrice
  }));
}

// Step 10: Process Payment capture via PaymentService
const paymentService = await cds.connect.to('PaymentService');
try {
  const paymentResult = await paymentService.createPayment({
    orderId,
    amount: grandTotal,
    currency: 'USD',
    paymentMethod
  });

  // Step 11: Payment success: commit inventory & update order
  for (const item of items) {
    await tx.run(UPDATE('sap.marketplace.Inventories')
      .set({
        quantityAvailable: { '-=': item.quantity },
        quantityReserved: { '-=': item.quantity }
      })
      .where({ variant_ID: item.variant_ID }));
  }

  await tx.run(UPDATE('sap.marketplace.Orders')
    .set({ orderStatus: 'CONFIRMED', paymentStatus: 'PAID' })
    .where({ ID: orderId }));

  // Step 12: Clear cart items & send customer notification
  await tx.run(DELETE.from('sap.marketplace.CartItems').where({ cart_ID: cartId }));
  await tx.run(INSERT.into('sap.marketplace.Notifications').entries({
    customer_ID: customer.ID,

```

```
        title: 'Order Confirmed',
        message: `Your order ${orderNumber} has been received and confirmed.`,
        isRead: false
    }));

    return { orderId, orderNumber, status: 'CONFIRMED', totalAmount: grandTotal };

} catch (err) {
    // Step 13: Payment failed: Rollback inventory hold & reject
    for (const item of items) {
        await tx.run(UPDATE('sap.marketplace.Inventories')
            .set('quantityReserved = quantityReserved - ', item.quantity)
            .where({ variant_ID: item.variant_ID }));
    }
    await tx.run(UPDATE('sap.marketplace.Orders')
        .set({ orderStatus: 'PAYMENT_FAILED', paymentStatus: 'FAILED' })
        .where({ ID: orderId }));
    req.reject(402, `Payment transaction failed: ${err.message}`);
}
});
```

Key Junior Takeaways:

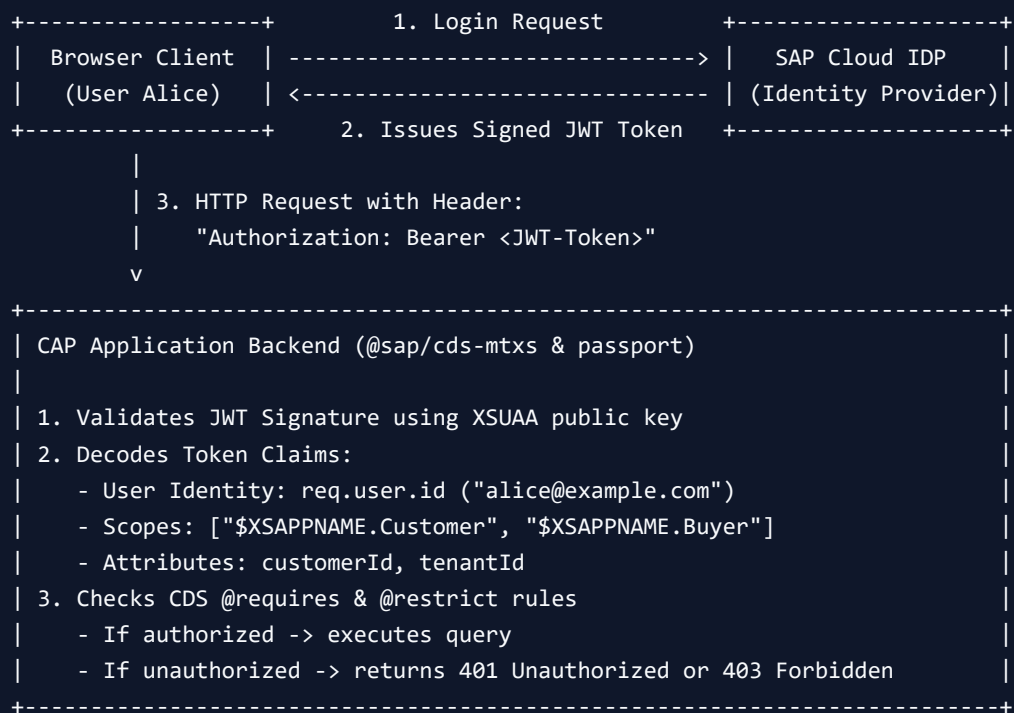
1. **Never perform multiple writes without `cds.tx(req)`** : If a network failure occurs midway, the transaction rollback prevents orphaned records or incorrect inventory deductions.
2. **Snapshot line items**: Notice step 9 writes `unitPrice` and `totalPrice` into `OrderItems` . Never calculate historical orders dynamically from current product prices.

Chapter 5: Security, Authentication & Authorization (XSUAA)

5.1 Architecture of SAP BTP Security

In enterprise cloud applications, security must never rely exclusively on frontend UI controls (such as hiding a "Delete" button). A malicious user can open Chrome DevTools or curl and submit HTTP requests directly to your backend APIs.

In SAP BTP, security is governed by the **Extended Services for User Account and Authentication (XSUAA)** service, which implements the **OAuth 2.0 / JWT (JSON Web Token)** standard.



5.2 Decoupling Roles and Scopes in `xs-security.json`

Open `xs-security.json` in the project root. It defines three key levels of security metadata:

1. Scopes (Technical Permissions)

Scopes are the granular technical permissions verified by the CAP runtime:

```
"scopes": [
  { "name": "$XSAPPNAME.Customer", "description": "Customer storefront access" },
  { "name": "$XSAPPNAME.Seller", "description": "Seller portal access" },
  { "name": "$XSAPPNAME.ProductManager", "description": "Manage catalog products" },
  { "name": "$XSAPPNAME.OrderManager", "description": "Manage customer orders & fulfillment" },
  { "name": "$XSAPPNAME.InventoryManager", "description": "Manage warehouse stock" },
  { "name": "$XSAPPNAME.Administrator", "description": "Full system administrator access" }
]
```

2. Role Templates (Blueprints)

Role templates bundle one or more scopes together into a functional template:

```
"role-templates": [
  {
    "name": "CustomerRole",
    "description": "Storefront customer capabilities",
    "scope-references": ["$XSAPPNAME.Customer"]
  },
  {
    "name": "AdminRole",
    "description": "Full administrative privileges",
    "scope-references": ["$XSAPPNAME.Administrator"]
  }
]
```

3. Role Collections (User Assignment)

Role collections are assigned to actual human users in the SAP BTP Cockpit:

```
"role-collections": [
  {
    "name": "Marketplace_Customer",
    "description": "Customer shoppers",
    "role-template-references": ["$XSAPPNAME.CustomerRole"]
  }
]
```

5.3 Enforcing Authorization in CDS (`@requires` & `@restrict`)

CAP provides two declarative annotations for service and entity authorization:

1. Service-Level Gatekeeping (`@requires`)

In `srv/admin-service.cds` :

```
service AdminService @(requires: ['Administrator', 'ProductManager', 'OrderManager', 'Inventory
```

* If an unauthenticated user or a standard customer attempts to query `/odata/v4/admin/Products`, CAP immediately rejects the request with **HTTP 403 Forbidden** before any database query is executed.

2. Entity-Level Operations & Row-Level Security (`@restrict`)

In `srv/customer-service.cds`:

```
service CustomerService @(requires: 'authenticated-user') {
  @restrict: [
    { grant: ['READ', 'UPDATE'], to: 'Customer', where: 'customer.email = $user.id' },
    { grant: '*', to: 'Administrator' }
  ]
  entity Addresses as projection on db.Addresses;
}
```

Why this is revolutionary:

* **Horizontal Privilege Escalation Prevention:** If Alice's customer ID is `cust-1` and Bob's is `cust-2`, Alice cannot read Bob's addresses even if she knows Bob's UUID!

* **Zero Boilerplate:** The `where: 'customer.email = $user.id'` clause is automatically injected by CAP into the underlying SQL query:

```
SELECT * FROM Addresses WHERE customer_email = 'alice@example.com';
```

5.4 Mocking Security in Local Development (`package.json`)

In local development, you do not have an active BTP XSUAA cloud instance running. CAP provides a mock authentication provider configured in `package.json`:

```
"cds": {
  "requires": {
    "auth": {
      "kind": "mocked",
      "users": {
        "alice": {
          "roles": ["Customer"],
          "attr": { "id": "cust-001" }
        },
        "carol_prod": {
          "roles": ["ProductManager"]
        },
        "admin": {
          "roles": ["Administrator", "Customer", "OrderManager", "ProductManager"]
        }
      }
    }
  }
}
```

To test an endpoint as Alice, simply pass Basic Authentication header:

Authorization: Basic YWxpY2U6 (alice with empty password).

In production (**[production]**), CAP automatically switches to **"kind": "xsuaa"**.

Chapter 6: Storefront Frontend (SAPUI5 Freestyle)

6.1 Why SAPUI5 Freestyle for the B2C Storefront?

SAP provides two frontend paradigms:

1. **SAP Fiori Elements:** A metadata-driven UI framework that generates standard enterprise screens from CDS annotations.
2. **SAPUI5 Freestyle:** A component-based MVC (Model-View-Controller) framework offering full control over HTML5 DOM, custom CSS styling, complex client animations, and bespoke user flows.

For our consumer-facing Storefront (`app/shop-ui`), we chose **SAPUI5 Freestyle**. Consumer e-commerce requires:

- * An Amazon-style dark navigation header with search suggestions and dynamic cart badge.
- * Rich merchandising hero banners and carousels.
- * High-density product card grids with discount badges and star rating displays.
- * Multi-step consumer checkout workflows.

6.2 Component & Manifest Configuration (`manifest.json`)

The `manifest.json` file is the application descriptor in SAPUI5. It defines data sources, models, and client-side routing.

1. Multiple OData V4 DataSources

Notice how `app/shop-ui/webapp/manifest.json` connects to our microservices:

```

"dataSources": {
  "catalogService": {
    "uri": "/odata/v4/catalog/",
    "type": "OData",
    "settings": { "odataVersion": "4.0" }
  },
  "cartService": {
    "uri": "/odata/v4/cart/",
    "type": "OData",
    "settings": { "odataVersion": "4.0" }
  },
  "orderService": {
    "uri": "/odata/v4/order/",
    "type": "OData",
    "settings": { "odataVersion": "4.0" }
  },
  "customerService": {
    "uri": "/odata/v4/customer/",
    "type": "OData",
    "settings": { "odataVersion": "4.0" }
  }
}

```

Each data source is instantiated into a named model:

- * Default Model (`"`): Bound to `catalogService` (for browsing products, categories, search).
- * `"cart"` : Bound to `cartService` (for shopping cart items and badge counter).
- * `"order"` : Bound to `orderService` (for order history and checkout).
- * `"customer"` : Bound to `customerService` (for address book and wishlist).

2. Client-Side Hash-Based Routing (17 Routes)

SAPUI5 uses declarative routing:

```

"routes": [
  { "name": "home", "pattern": "", "target": "home" },
  { "name": "productDetails", "pattern": "product/{productId}", "target": "productDetails" },
  { "name": "cart", "pattern": "cart", "target": "cart" },
  { "name": "checkout", "pattern": "checkout", "target": "checkout" },
  { "name": "orders", "pattern": "orders", "target": "orders" }
]

```

When the browser URL hash changes to `#/product/prod-001`, the router automatically displays the `ProductDetails.view.xml` view and triggers `ProductDetails.controller.js`.

6.3 Critical SAPUI5 OData V4 Binding Gotcha

WARNING:

The Paging Trap: Never put `$top` or `$skip` inside `parameters: { ... }` in SAPUI5 OData V4!

The Bug That Broke the Storefront:

In `Home.view.xml`, our featured product grid was originally declared like this:

```
<!-- WRONG - DO NOT DO THIS IN ODATA V4! -->
<grid:CSSGrid items="{
  path: '/Products',
  parameters: {
    '$filter': 'isFeatured eq true',
    '$top': 4,
    '$skip': 0
  }
}">
```

Why It Failed:

In SAPUI5 OData V4, paging is managed internally by the `ODataListBinding` class. If you pass `$top` or `$skip` inside `parameters`, the UI5 framework throws a fatal, unhandled exception:

```
Error: System query option $top is not supported - sap.ui.model.odata.v4.ODataListBinding
```

This crashed the SAPUI5 router, aborting the view initialization before anything rendered on screen!

The Correct SAPUI5 OData V4 Pattern:

Declare `length` and `startIndex` as top-level properties on the binding definition:

```
<!-- CORRECT - Standard SAPUI5 OData V4 Binding -->
<grid:CSSGrid items="{
  path: '/Products',
  length: 4,
  startIndex: 0,
  parameters: {
    '$filter': 'isFeatured eq true',
    '$expand': 'variants($expand=offers),images'
  }
}">
```

In addition, configure `"autoExpandSelect": false` in `manifest.json` for freestyle models to prevent UI5 from rewriting freestyle XML view bindings into malformed requests.

6.4 Reusable UI Components: Fragments & BaseController

1. XML Fragments for DRY (Don't Repeat Yourself) UI

In `app/shop-ui/webapp/view/fragments/`, we built modular UI components:

- * `ProductCard.fragment.xml`: Encapsulates the product thumbnail, price badge, rating stars, and "Add to Cart" button. Reused across Home, Search Results, and Category pages.
- * `CartSummary.fragment.xml`: Displays subtotal, tax, shipping, coupon input, and "Proceed to Checkout" button. Reused in Cart and Checkout views.

2. BaseController Pattern

Instead of duplicating helper methods across 17 controllers, `BaseController.js` provides common utilities:

```
sap.ui.define([
    "sap/ui/core/mvc/Controller",
    "sap/ui/core/UIComponent",
    "marketplace/shop/model/formatter"
], function (Controller, UIComponent, formatter) {
    "use strict";

    return Controller.extend("marketplace.shop.controller.BaseController", {
        formatter: formatter,

        getModel: function (sName) {
            return this.getView().getModel(sName) || this.getOwnerComponent().getModel(sName);
        },

        getRouter: function () {
            return UIComponent.getRouterFor(this);
        },

        navTo: function (sRoute, mData) {
            this.getRouter().navTo(sRoute, mData);
        }
    });
});
```

Chapter 7: Backoffice Frontend (SAP Fiori Elements)

7.1 What is SAP Fiori Elements?

For the internal business backoffice (`app/admin-ui`), our requirements were very different from the consumer storefront:

- * Business users need comprehensive management screens for 9 operational areas: Products, Orders, Customers, Inventory, Sellers, Promotions, Reviews, Payments, and Shipments.
- * Building 9 separate CRUD applications from scratch in freestyle UI5 would take months and produce inconsistent UI layouts.

SAP Fiori Elements solves this by generating complete enterprise web applications entirely from **CDS UI Annotations**. Instead of writing HTML/XML views and JavaScript controllers, we write declarative metadata in `app/admin-ui/annotations.cds`.

7.2 Core CDS UI Annotations Breakdown (`app/admin-ui/annotations.cds`)

Let us examine the annotations that power the **Product Management** screen:

1. Header Information (`@UI.HeaderInfo`)

Defines the title of the page and singular/plural entity names:

```
annotate AdminService.Products with @(
  UI.HeaderInfo: {
    TypeName: 'Product',
    TypeNamePlural: 'Products',
    Title: { $Type: 'UI.DataField', Value: title },
    Description: { $Type: 'UI.DataField', Value: brand }
  }
);
```

2. List Report Columns (`@UI.LineItem`)

Defines the table columns, their display order, and responsiveness:

```

annotate AdminService.Products with @(
  UI.LineItem: [
    { $Type: 'UI.DataField', Value: title, Label: 'Product Title', ![@UI.Importance]:
    { $Type: 'UI.DataField', Value: brand, Label: 'Brand' },
    { $Type: 'UI.DataField', Value: category.name, Label: 'Category' },
    { $Type: 'UI.DataField', Value: basePrice, Label: 'Base Price' },
    { $Type: 'UI.DataField', Value: averageRating, Label: 'Rating' },
    {
      $Type: 'UI.DataField',
      Value: status,
      Label: 'Status',
      Criticality: statusCriticality
    }
  ]
);

```

3. Filter Bar (`@UI.SelectionFields`)

Defines which fields appear in the SmartFilterBar at the top of the table:

```

annotate AdminService.Products with @(
  UI.SelectionFields: [
    title,
    brand,
    category_ID,
    seller_ID,
    status
  ]
);

```

Fiori Elements automatically renders input fields, date pickers, or value help dialogs based on the underlying data types.

4. Semantic Colors (`@UI.Criticality`)

Criticality renders semantic color badges without custom CSS:

- * `1` or `#Negative` = Red (e.g. OUT_OF_STOCK, CANCELLED)
- * `2` or `#Critical` = Orange/Yellow (e.g. LOW_STOCK, PENDING_PAYMENT)
- * `3` or `#Positive` = Green (e.g. IN_STOCK, DELIVERED, ACTIVE)
- * `0` or `#Neutral` = Grey/Default

In `srv/admin-service.js`, we calculate `statusCriticality` dynamically in `after('READ')`:

```
this.after('READ', 'Products', (each) => {
  if (each.status === 'ACTIVE') each.statusCriticality = 3;
  else if (each.status === 'INACTIVE') each.statusCriticality = 1;
  else each.statusCriticality = 0;
});
```

5. Object Page Sections (`@UI.Facets` & `@UI.FieldGroup`)

When a user clicks a row in the List Report, Fiori Elements opens the **Object Page**.

`@UI.Facets` organizes the layout into tabs and accordion sections:

```
annotate AdminService.Products with @(
  UI.Facets: [
    {
      $Type: 'UI.CollectionFacet',
      ID: 'GeneralSection',
      Label: 'General Information',
      Facets: [
        { $Type: 'UI.ReferenceFacet', Target: '@UI.FieldGroup#General', Label: 'De'
        { $Type: 'UI.ReferenceFacet', Target: '@UI.FieldGroup#Pricing', Label: 'Pr
      ]
    },
    {
      $Type: 'UI.ReferenceFacet',
      Target: 'variants/@UI.LineItem',
      Label: 'Product Variants'
    },
    {
      $Type: 'UI.ReferenceFacet',
      Target: 'reviews/@UI.LineItem',
      Label: 'Customer Reviews'
    }
  ],
  UI.FieldGroup#General: {
    Data: [
      { $Type: 'UI.DataField', Value: title },
      { $Type: 'UI.DataField', Value: description },
      { $Type: 'UI.DataField', Value: brand }
    ]
  }
);
```

* Notice how `variants/@UI.LineItem` reuses the variant entity's line items table as a nested child table on the product screen!

6. Search Helps (`@Common.ValueList`)

When selecting a category, users should see a search dialog rather than typing raw UUIDs:

```
annotate AdminService.Products with {  
  category @Common.ValueList: {  
    CollectionPath: 'Categories',  
    Parameters: [  
      { $Type: 'Common.ValueListParameterInOut', LocalDataProperty: category_ID, ValueListProperty: 'name' }  
      { $Type: 'Common.ValueListParameterDisplayOnly', ValueListProperty: 'name' }  
    ]  
  }  
};
```

Chapter 8: Multitarget Application (MTA) & Cloud Deployment

8.1 What is an SAP Multitarget Application (MTA)?

In enterprise cloud development, a complete business application is rarely a single monolithic server. Our marketplace consists of:

1. A Node.js CAP backend service.
2. A SAP HANA Cloud relational database schema.
3. Two independent HTML5 frontend web applications (`shop-ui` and `admin-ui`).
4. Multiple cloud service bindings (XSUAA, Destinations, HTML5 App Repo, HDI).

In SAP BTP, deploying these distinct pieces independently by hand would be error-prone and slow.

The **Multitarget Application (MTA)** standard solves this by packaging the entire distributed system into a single cohesive archive (`.mtar`) governed by a single deployment descriptor: `mta.yaml` .

8.2 Dissecting `mta.yaml`

The `mta.yaml` file defines **Modules** (software artifacts that run) and **Resources** (cloud services provisioned on BTP):

```
+-----+
|                                     mta.yaml ROOT ARCHIVE                                     |
|                                                                                             |
| MODULES (What runs):                                                                    |
| 1. amazon-clone-srv                (Node.js CAP Backend)                               |
| 2. amazon-clone-db-deployer        (HANA Database Migrations Deployer)                 |
| 3. amazon-clone-app-deployer       (HTML5 Application Repository Content Deployer)      |
| 4. marketplace-shop                (SAPUI5 Storefront Frontend)                       |
| 5. marketplace-admin               (SAP Fiori Elements Admin Frontend)                |
|                                                                                             |
| RESOURCES (BTP Managed Services):                                                         |
| - amazon-clone-db                  (SAP HANA HDI Container)                           |
| - amazon-clone-auth                (SAP BTP XSUAA Service Instance)                   |
| - amazon-clone-destination          (BTP Destination Service)                        |
| - amazon-clone-html5-repo-host      (HTML5 Application Repository)                    |
|                                                                                             |
+-----+
```

Key Module Configurations:

1. Backend Service Module (`amazon-clone-srv`)

```
- name: amazon-clone-srv
  type: nodejs
  path: gen/srv
  parameters:
    buildpack: nodejs_buildpack
    memory: 512M
  provides:
    - name: srv-api
      properties:
        srv-url: ${default-url}
  requires:
    - name: amazon-clone-db
    - name: amazon-clone-auth
    - name: amazon-clone-destination-service
```

* Notice `provides: srv-api` : This exports the backend's live URL so other modules and destinations can bind to it dynamically.

2. HANA Database Deployer Module (`amazon-clone-db-deployer`)

```
- name: amazon-clone-db-deployer
  type: hdb
  path: gen/db
  requires:
    - name: amazon-clone-db
```

* Uses SAP's official **HDB Deployer** container to run database migrations, create SAP HANA column tables, and create database views automatically.

8.3 The Build and Packaging Pipeline

To produce the deployment archive, run the following automated pipeline:

```
<h1 class="chapter-title">1. Compile CDS models to production output (gen/srv and gen/db)</h1>
npx cds build --production

<h1 class="chapter-title">2. Package both UI5 applications into zip archives</h1>
npm run build:ui

<h1 class="chapter-title">3. Compile the MTA Archive using Cloud MTA Build Tool (mbt)</h1>
mbt build
```

The output is written to:

```
mta_archives/amazon-clone_1.0.0.mtar
```

8.4 Production Deployment Commands (Cloud Foundry)

Once the `.mtar` file is generated, deploy it to your SAP BTP subaccount:

```
<h1 class="chapter-title">Step 1: Login to SAP BTP Cloud Foundry API</h1>
cf login -a https://api.cf.eu10.hana.ondemand.com

<h1 class="chapter-title">Step 2: Target your Organization and Space</h1>
cf target -o my-btp-org -s dev

<h1 class="chapter-title">Step 3: Deploy the MTA archive</h1>
cf deploy mta_archives/amazon-clone_1.0.0.mtar
```

The Cloud Foundry deployer reads `mta.yaml`, automatically creates all service instances (XSUAA, HDI Container, Destination Service), deploys the database schema, launches the Node.js backend, and uploads the HTML5 applications into the SAP BTP HTML5 Repository.

Chapter 9: Enterprise Automated Testing & Quality Assurance

9.1 The Testing Philosophy in SAP CAP

In enterprise applications, manual testing is insufficient. A change in a single CDS entity or calculation handler can ripple across multiple downstream services.

Our project features **136 automated tests across 9 comprehensive test suites**, achieving 100% pass rate.

We use the official SAP testing harness `@cap-js/cds-test`, combined with **Mocha** and **Chai**:

```
import cds from '@sap/cds';  
const { GET, POST, expect } = cds.test(__dirname + '/../..');
```

Why `@cap-js/cds-test` is Powerful:

- * Automatically spins up a real CAP test server with an in-memory database.
- * Deploys the entire `db/schema.cds` and populates seed data from `db/data/` in milliseconds.
- * Provides high-level HTTP client helpers (`GET` , `POST` , `PATCH` , `DELETE`) with built-in mock authentication:

```
// Send request as Alice (Customer role)  
const response = await GET('/odata/v4/cart/Carts', { auth: { username: 'alice' } });
```

9.2 The 9 Test Suites Breakdown

Test Suite	File Path	Focus Area
1. Catalog Tests	<code>test/catalog.test.js</code>	Category hierarchy, product search, brand filters, active status
2. Cart Tests	<code>test/cart.test.js</code>	Add to cart, quantity change, tax & shipping calculations, coupons
3. Checkout & Orders	<code>test/checkout-order.test.js</code>	18-step checkout, stock reservation, cancellations, customer returns
4. Customer Service	<code>test/customer-service.test.js</code>	Profile, addresses, wishlist, notifications, data isolation
5. Inventory & Admin	<code>test/inventory-admin.test.js</code>	Real-time stock counts, reservations, warehouse sync, shipments
6. Payment Service	<code>test/payment-service.test.js</code>	Payment capture, simulated card gateway, refunds, idempotency
7. Security & XSUAA	<code>test/auth-xsuaa.test.js</code>	401 unauthenticated, 403 forbidden, role-based access control
8. Negative & Edge Cases	<code>test/negative-scenarios.test.js</code>	Insufficient stock, invalid coupons, empty cart checkout
9. Critical E2E Flow	<code>test/e2e-critical-flow.test.js</code>	12-Step sequential user journey simulating real shopper

9.3 The 12-Step End-to-End Critical User Journey

In `test/e2e-critical-flow.test.js`, we execute a complete sequential business flow:

```

1. Browse Catalog --> 2. Search Product --> 3. Open Details --> 4. Add to Cart
    |
    v
5. Change Quantity --> 6. Apply Coupon --> 7. Checkout --> 8. Payment Capture
    |
    v
9. Order Created --> 10. View History --> 11. Cancel Order --> 12. Submit Review

```

Real Test Code Example:

```
it('should execute 18-step checkout successfully', async () => {
  const { data } = await POST('/odata/v4/order/checkout', {
    cartId: 'cart0000-0000-0000-0000-000000000001',
    shippingAddressId: 'addr0000-0000-0000-0000-000000000001',
    billingAddressId: 'addr0000-0000-0000-0000-000000000001',
    paymentMethod: 'CREDIT_CARD',
    couponCode: 'WELCOME10'
  }, { auth: { username: 'alice' } });

  expect(data.status).toEqual('CONFIRMED');
  expect(data.orderNumber).toMatch(/^ORD-/);
  expect(data.totalAmount).toBeGreaterThan(0);
});
```

9.4 Running Tests and Linting

Every junior developer must run these commands before submitting code:

```
<h1 class="chapter-title">Run all 136 tests:</h1>
npm test

<h1 class="chapter-title">Run CDS code style and syntax linter:</h1>
npm run lint
```

Chapter 10: Junior Developer Playbook & Common Pitfalls

10.1 Ten Common Mistakes Every Junior CAP Developer Makes

Mistake 1: Unexposed Navigation Targets with `$expand`

* **The Error:** `[400] Bad Request: Navigation property "xyz" does not exist`.

The Cause: You called `$expand=xyz` from the UI, but entity `xyz` was not declared inside that specific service in `srv/.cds`.

* **The Fix:** Always expose a `@readonly entity xyz as projection on db.xyz;` in the service.

Mistake 2: Declaring `$top` / `$skip` in SAPUI5 Binding Parameters

* **The Error:** `System query option $top is not supported`.

* **The Cause:** SAPUI5 OData V4 manages paging internally. Putting `$top` in XML view `parameters` crashes the router.

* **The Fix:** Put `length: 4` and `startIndex: 0` directly on the binding object outside `parameters`.

Mistake 3: Executing Multiple Database Writes Without a Transaction

* **The Error:** Data corruption when a failure occurs midway through checkout.

* **The Cause:** Calling `await INSERT` and `await UPDATE` without binding to `cds.tx(req)`.

* **The Fix:** Always use `const tx = cds.tx(req); await tx.run(...);`. If an error occurs, CAP automatically rolls back all changes.

Mistake 4: Using Association Instead of Composition for Line Items

* **The Error:** Child records are orphaned when parent is deleted, or deep insert fails.

* **The Cause:** Using `items : Association to many OrderItems` instead of `items : Composition of many OrderItems on items.order = $self`.

* **The Fix:** Use `Composition of` for parent-child lifecycles.

Mistake 5: Incorrect CSV Seed File Headers

* **The Error:** Foreign key values are null or ignored when loading mock data.

* **The Cause:** Naming the CSV column `category` instead of `category_ID`.

* **The Fix:** For any association `category : Association to Categories`, the relational column in SQL and CSV is always `_ID`.

Mistake 6: Relying on Frontend Hiding for Security

* **The Error:** Unauthorized users can delete records or view other customers' carts using curl or Postman.

* **The Cause:** Checking user role in JavaScript UI instead of CDS `@restrict`.

* **The Fix:** Always enforce security at the CAP service layer with `@requires` and `@restrict`.

Mistake 7: Hardcoding Full Cloud URLs

* **The Error:** Frontend fails when deployed to Cloud Foundry due to CORS or broken domains.

* **The Cause:** Hardcoding `http://localhost:4004` in UI5 models or manifest.

* **The Fix:** Use relative paths (e.g., `/odata/v4/catalog/`) and route via the Managed App Router.

Mistake 8: Forgetting to Re-Deploy SQLite Views

* **The Error:** `SQLITE_ERROR: no such table: CartService_Products`.

* **The Cause:** Adding a new service projection without redeploying the SQLite schema.

The Fix: Run `npx cds deploy --to sqlite:db.sqlite` or `cds.deploy('')`.

Mistake 9: Failing to Freeze Historical Snapshot Prices

* **The Error:** An order placed last month changes total price when the seller raises the catalog price today.

* **The Cause:** Calculating order totals by joining with current catalog product prices.

* **The Fix:** Snapshot the price into `OrderItems.unitPrice` at the exact second of purchase.

Mistake 10: Writing Custom UI Code Instead of Annotations in Admin Screens

* **The Error:** Huge boilerplate JavaScript controllers for simple tabular and form screens.

* **The Cause:** Not knowing how to leverage Fiori Elements.

* **The Fix:** Use `@UI.LineItem`, `@UI.HeaderInfo`, and `@UI.Facets`. Fiori Elements handles sorting, filtering, paging, and responsive rendering with 0 JavaScript.

10.2 Essential CDS CLI Cheat Sheet

Command	What It Does
<code>cds watch</code>	Starts local development server with auto-restart on file save
<code>cds serve</code>	Serves all services in <code>srv/</code> on port 4004
<code>cds compile db/schema.cds --to sql</code>	Inspects the generated SQL DDL statements
<code>cds compile srv/catalog-service.cds -to edmx</code>	Inspects the generated OData V4 EDMX XML metadata
<code>cds deploy --to sqlite:db.sqlite</code>	Deploys schema and views to local SQLite database
<code>cds lint</code>	Validates CDS syntax, naming conventions, and best practices
<code>cds env</code>	Inspects active configuration, database profiles, and auth settings
<code>npm test</code>	Runs the entire 136-test suite
<code>npm run build</code>	Runs production compilation and UI packaging
<code>mbt build</code>	Builds the complete Cloud Foundry <code>.mtar</code> deployment archive

10.3 Step-by-Step: How to Add a New Feature to This Project

Follow this 7-step checklist whenever you are assigned a new business requirement:

```
Step 1: Declare Domain Model in db/schema.cds
      (Use cuid, managed, enums, Associations/Compositions)
      |
      v
Step 2: Add Seed CSV Data in db/data/sap.marketplace-<Entity>.csv
      (Include header row with UUIDs and _ID foreign keys)
      |
      v
Step 3: Expose Projection in srv/<service-name>.cds
      (Add @readonly, @restrict, virtual fields, actions)
      |
      v
Step 4: Implement Logic in srv/<service-name>.js
      (Handle before validation, on actions, after virtual fields)
      |
      v
Step 5: Bind to UI
      (Freestyle XML View for Storefront OR CDS Annotations for Admin)
      |
      v
Step 6: Write Automated Tests in test/
      (Test happy path + negative edge cases with @cap-js/cds-test)
      |
      v
Step 7: Verify with npm test and npm run lint
      (Ensure all 136+ tests pass with 0 errors)
```